

Applications of Statistical Modeling:

Module #3, Lecture #1

Classification Trees: Overview and Software

Instructor: Brady T. West

Email: bwest@umich.edu

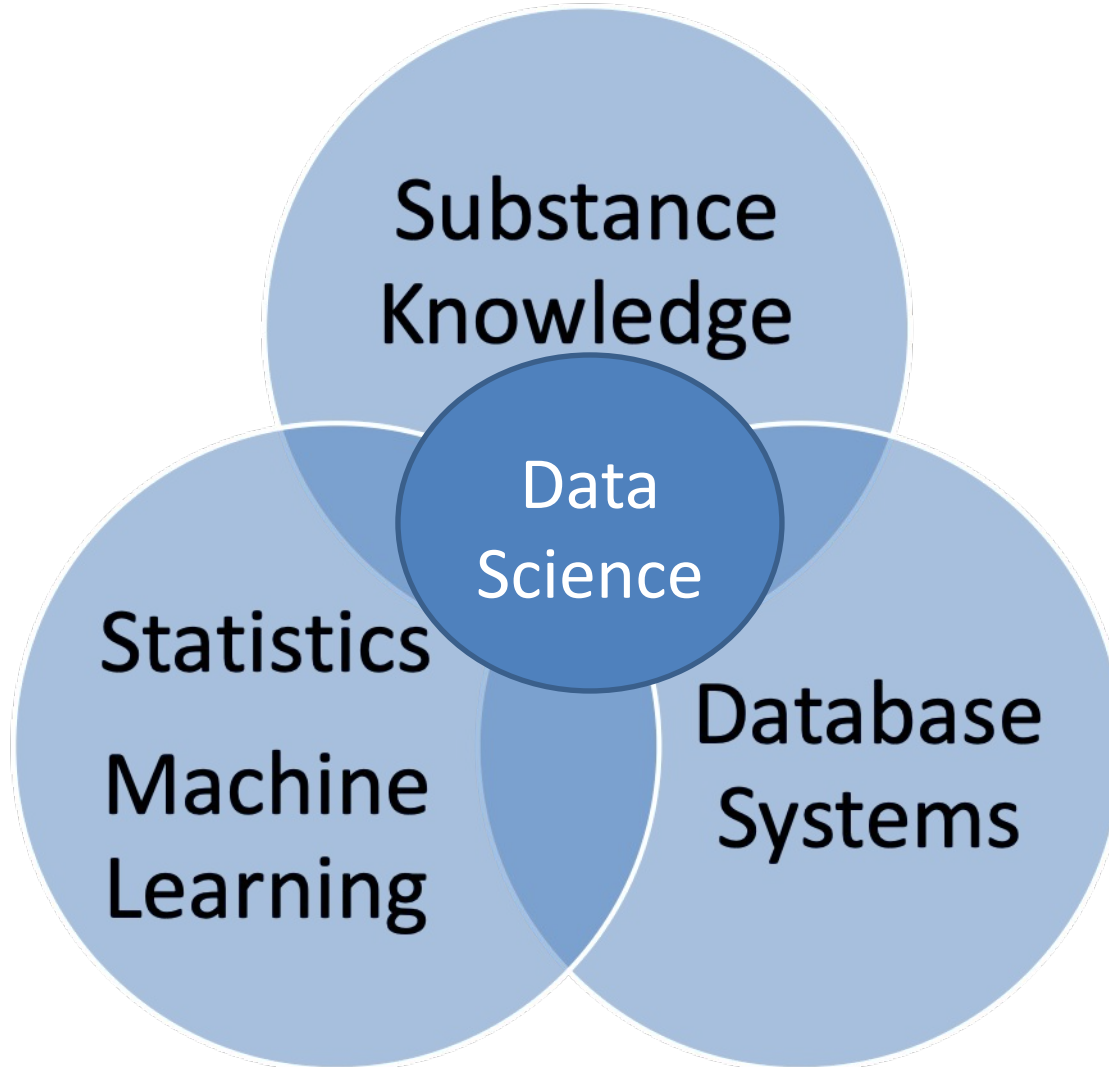
A Shift: Inference → Prediction

- With the start of this third module, we are now shifting from modeling approaches that focus on inference (MLM, GEE, SEM, etc.)...
- ...to more algorithmic approaches that focus on optimizing predictive power
- This will be a general two-week introduction to the ideas of machine learning methods

Two Types of Learning Tasks

- **Prediction:** Use existing data to predict unknown or future values, such as
 - Regression (predict means)
 - Classification (predict categories; today's focus)
- **Description:** Find human-interpretable patterns that describe the data, such as
 - Visualization
 - Association analysis
 - Clustering

Origins of Data Science



Artificial Intelligence (AI)

- **Definition:**
 - AI refers to the simulation of human intelligence processes by machines, especially computer systems.
- **Key Concepts:**
 - Learning from data
 - Adapting over time
 - Making decisions
- **Examples in Daily Life:**
 - Smart assistants
 - Chatbots
 - Ecommerce
 - Autonomous vehicles

Machine Learning 1

Definition:

- Machine Learning (ML) is a subset of AI that involves training algorithms to make predictions or decisions based on data.

Types of Machine Learning:

- **Supervised Learning:** Uses labeled data to train models (e.g., classification, regression)
- **Unsupervised Learning:** Finds patterns in unlabeled data (e.g., clustering, association)
- **Reinforcement Learning:** Models learn by receiving rewards or penalties based on actions taken

Workflow:

- Data Collection
- Data Preprocessing
- Model Training
- Model Evaluation

Machine Learning 2

- A “hot topic” as of late in statistics, data science, and survey methodology
- Rather than testing hypotheses, we “mine” a large set of data to “learn” interesting patterns that could be used to predict future outcomes
- Very important in our current era, where we have access to large data sets and powerful computing tools that can be used to discover these types of interesting patterns

Natural Language Processing (NLP)

- **Definition:**
 - NLP is a subfield of AI focused on the interaction between computers and humans through natural language.
- **Key Techniques:**
 - **Tokenization:** Breaking text into words or sentences
 - **Stop-word Removal:** Removing common words (e.g., "the", "and") to focus on important terms
 - **Stemming/Lemmatization:** Reducing words to their base or root form
 - **Sentiment Analysis:** Determining the sentiment expressed in text
- **Applications:**
 - Sentiment Analysis of social media data
 - Chatbots and Virtual Assistants
 - Machine Translation

Deep Learning

- **Definition:**
 - Deep learning as a subset of machine learning focusing on neural networks
- **Concepts:**
 - **Neurons:** Basic units that receive input and perform a weighted sum followed by an activation function
 - **Neural Networks:** a series of algorithms that attempt to recognize underlying relationships in a set of data, inspired by how the human brain operates.
 - **Layers:** Input, hidden, and output layers
 - **Activation Functions:** Sigmoid, Rectified Linear Unit-ReLU, etc.
- **Types:**
 - **Feedforward Neural Networks:** Simple, wherein connections do not form cycles
 - **Convolutional Neural Networks (CNNs):** Primarily used for image data
 - **Recurrent Neural Networks (RNNs):** Used for sequential data like text or time series

Applications of AI in Survey Research

- **Automated Data Cleaning:**
 - AI algorithms can automatically detect and correct data entry errors, handle missing values, and standardize data formats.
 - Example Tool: janitor package in R.
- **Predictive Analytics:**
 - AI models can predict future trends and behaviors based on survey data.
 - Example: Predicting customer satisfaction based on survey responses.
- **Natural Language Processing:**
 - AI can analyze open-ended survey responses to detect sentiment, themes, and patterns.
 - Example Tool: sentimentr package in R.
- **Adaptive Survey Designs:**
 - AI can tailor survey questions based on respondent behavior, increasing the relevance and accuracy of the data collected.

Overview of Today's Lecture

- Why classification trees and machine learning?
- Essential concepts underlying classification trees
- Techniques for building classification trees
- Concepts and techniques for random forests
- Available software procedures
- A very cool visual introduction to the tree-building process: [A visual explanation of the tree-building process.](#)

Classification Trees

- A widely used tool for machine learning
- Unlike standard logistic regression models for categorical dependent variables, classification trees enable **automatic detection** of important **interactions** between predictors
- **Important question:** Do classification trees improve on the ability of “classical” logistic regression models to accurately predict categorical outcomes?

Classification Trees: Concepts 1

- The goal of classification trees is to partition sample cases up into **hypercubes** (similar “groups” in higher dimensions), and assign a class (outcome) to each hypercube
- We want to split the sample space in such a way that most members belonging to a class (e.g., respondents) fall in the same hypercube
- We will discuss **binary trees**, where **nodes** of the tree may split into two **branches**

Classification Trees: Concepts 2

- Nothing prevents a tree from assigning one case to each hypercube...a “perfect” fit
- We need to think about how to **construct** and **prune** a given tree to yield a reasonable classification solution that is not trivial
- We arrive at solutions by iteratively identifying the next best split at each point in the tree, and optimizing the splits for classification
 - We don’t optimize the entire tree all at once

Classification Trees: Concepts 3

- One possible measure for choosing splits in this “one-step look-ahead” approach is the **deviance** measure
- At each node v of the tree, we have a probability distribution across the classes, denoted by $P(v,k)$, where k is a class
- We **condition** on the observed predictors X for the cases in the data set to **learn** the case count, n_{vk} , for each class in each node

Classification Trees: Concepts 4

- We can then define the deviance associated with a given node (via the conditional likelihood) as

$$D_v = -2 \sum_{k=1}^K n_{vk} \log(P(v, k))$$

And for the tree as a whole as

$$D = \sum_v D_v$$

- When considering splitting a node v into nodes $v1$ and $v2$, we choose the split that maximizes the reduction in deviance $D_v - D_{v1} - D_{v2}$

Classification Trees: Concepts 5

- Two main problems with this basic approach:
 1. Overfitting (can't determine whether changes in deviance are *significant*)
 2. A bias toward predictors with many possible values
- A better approach: **conditional inference trees** constructed using **permutation tests**
 - Implemented in `ctree` within the `partykit` package in R
- Permutation tests (Strasser and Weber, 1999, *MMS*) are used to determine the statistical significance of a potential split (only split using variables strongly related to the outcome!)
- See [Vignettes for the party package](#) for additional technical details

Classification Trees: Concepts 6

- There are two types of errors that we can discuss when evaluating solutions:
 - **Apparent error rate:** The error rate of our tree when applied to the **training data** T (typically computed using a **confusion matrix**, showing observed vs. predicted classes for sample cases)
 - What % of cases are off the diagonal?
 - **True error rate:** The error rate of our tree when applied to **new data** (typically called a **test** set)

Classification Trees: Concepts 7

- We could assess the true error rate by splitting a data set into training and test subsets (given that the classes will be known for the test data set, enabling validation)
- Note that this wastes data that could be used for training and constructing the tree!
- We can also use **cross-validation** as a strategy for assessing error rates

Classification Trees: Concepts 8

- **Steps in Cross-Validation:**
 1. Hold out a fraction (say 5-10%) of the training data set denoted by T (could even leave out 1 observation!)
 2. Use the remaining data to construct a given tree
 3. Apply the tree to the “hold out” (or “out of bag”) sample, and estimate the true error rate
 4. Now, leave out a different fraction of T , and repeat the second and third steps
- The **cross-validated error rate** will be the average of the error rates estimated in Step 3 (there will be variable error rates, given many hold out samples!)

Aggregating Trees

- **Idea:** Significant gains in classification accuracy are possible by aggregating the resulting rules of trees (which can tend to be unstable) built from perturbed versions of the same data set (Breiman, 1996)
- We could form **bootstrap replicates** of the same data set, using with-replacement selection of samples of the same size (basic idea of **random forests**)
- The multiple solutions are aggregated using **plurality voting**: use the modal classification assigned to each case in the data set, *across trees*

An Example Using `ctree()`

- Let's consider an example to illustrate the use of the `partykit` package in R
- We'll use the `GlaucomaM` data set, which comes with the `TH.data` package, and contains 62 predictors of Glaucoma in eyes based on laser scanning ($n = 196$):

```
library(partykit); data("GlaucomaM", package="TH.data")
```

- We'll first use the entire data set to fit a tree, and we start by setting control parameters

An Example using `ctree()`

- We first define control parameters, indicating that at least 6 objects must exist in a node for a split to be attempted, and that at least 3 objects must exist in every terminal node (**NOTE:** there are many other possible control options):

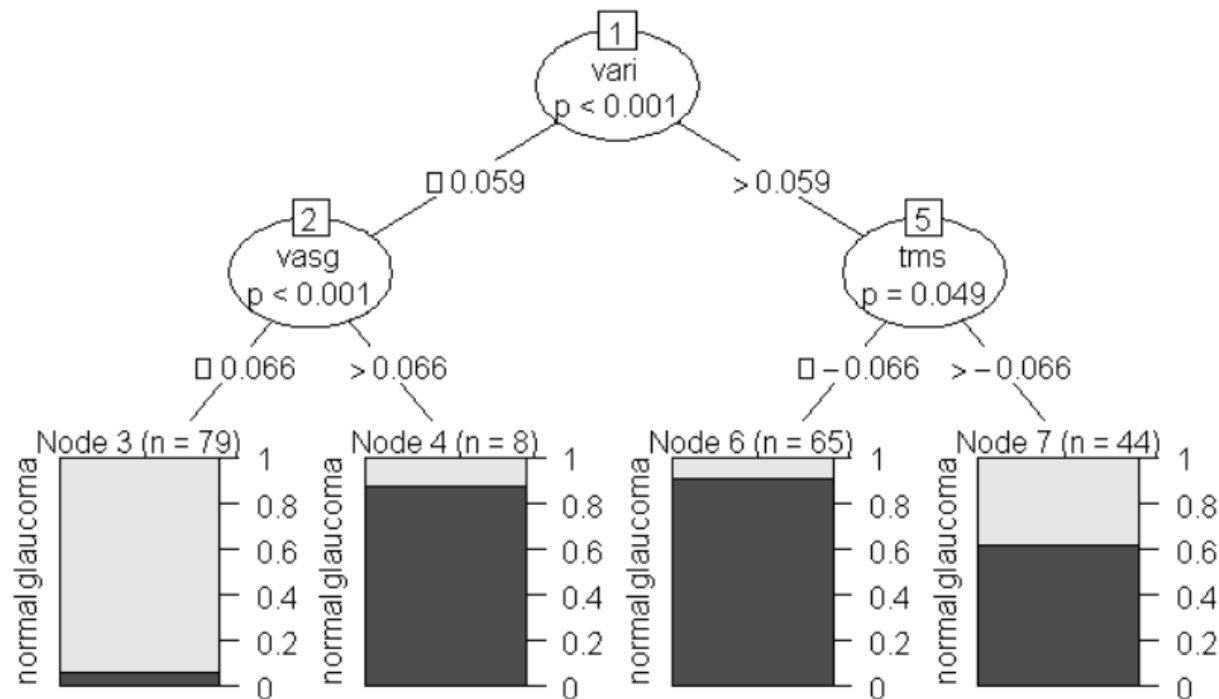
```
k.control <- ctree_control(minsplit=6, minbucket=3)
```

- Next, we build the classification tree using `ctree()`:

```
gtree <- ctree(Class ~ ., data = GlaucomaM,  
  control = k.control) # (~ . just means use all predictors)
```

An Example using `ctree()` 1

- We first examine the solution by plotting the tree: `plot(gtree)`



An Example using `ctree()` 2

- Note that this tree defines decision rules that we could follow when classifying cases
- We see evidence of three important predictors, and two interactions (`vari * vasg`, and `vari * tms`)!
- Let's examine the quality of the solution in more detail: `print(gtree)`

An Example using `ctree()` 3

Model formula:

```
Class ~ ag + at + as + an + ai + eag + eat + eas + ean + eai +  
      abrg + abrt + abrs + abrn + abri + hic + mhcg + mhct + mhcs +  
      mhcn + mhci + phcg + phct + phcs + phcn + phci + hvc + vbsg +  
      vbst + vbss + vbsn + vbsi + vasg + vast + vass + vasn + vasi +  
      vbrg + vbrr + vbrr + vbrr + vbrr + vbrr + vbrr + vbrr + vbrr +  
      vari + mdg + mdt + mds + mdn + mdi + tmg + tmt + tms + tmn +  
      tmi + mr + rnf + mdic + emd + mv
```

Fitted party:

```
[1] root  
|   [2] vari <= 0.059  
|   |   [3] vasg <= 0.066: glaucoma (n = 79, err = 6.3%)  
|   |   [4] vasg > 0.066: normal (n = 8, err = 12.5%)  
|   [5] vari > 0.059  
|   |   [6] tms <= -0.066: normal (n = 65, err = 9.2%)  
|   |   [7] tms > -0.066: normal (n = 44, err = 38.6%)
```

Number of inner nodes: 3

Number of terminal nodes: 4

We see decision rules and the resulting predictions that one would make in each of the four terminal nodes, along with apparent error rates (ranging from 6.3% to 38.6%).

The predictors `vasg` and `tms` only matter for specific values of `vari`, and we see a strong classification rule for glaucoma (`vari <= 0.059`, and `vasg <= 0.066`)!

An Example using `ctree()` 4

- Given the overall apparent error rates, let's dig into additional metrics for gauging apparent predictive ability
- FP = false positive
- TP = true positive
- FN = false negative
- TN = true negative
- We can calculate a variety of metrics for gauging predictive ability based on these counts

An Example using `ctree()` 5

- We calculate these quantities from a **confusion matrix**, which is a cross-tabulation of actual outcome values and predicted outcome values
- Then:
 - $\text{accuracy} = P(\text{accurate}) = (TP+TN)/(FP+TN+TP+FN)$
 - $\text{sensitivity} = P(\text{pred P} \mid \text{actual P}) = TP/(TP+FN)$
 - $\text{specificity} = P(\text{pred N} \mid \text{actual N}) = TN/(FP+TN)$
 - $\text{positive predictive value} = TP/(FP+TP)$
 - $\text{negative predictive value} = TN/(TN+FN)$
 - $\text{log odds ratio} = \log(TP*TN/(FN*FP))$

An Example using `ctree()` 6

- Continuing with our example:

```
library(crossval)
```

```
GlaucomaM$pred <- predict(gtree, GlaucomaM[, -63])
```

```
cm <- confusionMatrix(factor(GlaucomaM$Class),  
factor(GlaucomaM$pred), negative="normal")
```

```
cm
```

```
FP TP TN FN
```

```
5 74 93 24
```

```
diagnosticErrors(cm)
```

acc	sens	spec	ppv	npv	lor
0.8520408	0.7551020	0.9489796	0.9367089	0.7948718	4.0491728

Overall, good performance! But remember...this is apparent.

An Example using `ctree()` 7

- We have thus been able to construct a tree that allows us to estimate the **apparent error rate** (since we are using the full data set)
- Keep in mind that we have used fairly restrictive control parameters...
- Now, we consider an approach using **training** and **test** data to estimate the **true error rate**
- Drop predicted values in `GlaucomaM` first!

```
GlaucomaM <- GlaucomaM[, -64]
```

An Example using `ctree()` 8

- Code to create training and test data sets (with 70% of data being training):

```
set.seed(41279)
smp_size = floor(0.7 * nrow(GlaucomaM))
train_ind = sample(seq_len(nrow(GlaucomaM)), size =
  smp_size)
train = GlaucomaM[train_ind, ]
val = GlaucomaM[-train_ind, ] #test data
```

- Now, build the tree using the training set:

```
ttree <- ctree(Class ~ ., data = train, control =
  k.control)
```

An Example using `ctree()` 9

- Now, compute predictions in the test data, and compute metrics of predictive ability:

```
val$pred <- predict(ttrees, val[, -63])  
cm <- confusionMatrix(factor(val$Class), factor(val$pred),  
negative="normal")  
diagnosticErrors(cm)
```

acc	sens	spec	ppv	npv	lor
0.7627119	0.7575758	0.7692308	0.8064516	0.7142857	2.3434071

- Still good performance, but note the drop in specificity! Cross-validation involves repeating this process for different subsamples of the data (**folds**).

An Example using `ctree()` 10

- Some final thoughts on this example:
 - The conditional inference approach avoids the need to “trim” or “prune” trees, as only the most important predictors are identified (based on the permutation tests)
 - While one can write a program to perform cross-validation, the use of training and test data sets is generally sufficient in practice (assuming random selection of the training set)
 - The significance level for the tests can be adjusted using the control argument `mincriterion` (e.g., $0.95 \rightarrow p < 0.05$)

Regression Trees

- One can also fit **regression trees** to continuous outcome data using `ctree()`
- These trees are useful for computing predicted means as a function of complex interactions (errors now on continuous scale)
- Simply use a continuous dependent variable, instead of a categorical factor!

Other Software for Classification Trees

- The `rpart` package (older, widely used, but less effective than `ctree`)
- The WEKA software (freeware):
<http://www.cs.waikato.ac.nz/ml/weka/>
- The `rpms` package in R (specifically for complex sample survey data)
- Classification and Regression Tree (CART) Software (commercial, powerful, but expensive)
 - See Lewis (2000) for examples
- SAS Enterprise Miner (CHAID), and PROC HPSPLIT
(http://support.sas.com/documentation/cdl/en/stathpug/68163/HTML/default/viewer.htm#stathpug_hpsplit_examples01.htm)
- SPSS AnswerTree Software
- Stata: User-written command **-cart-**; type:
`net describe cart,`
`from (http://fmwww.bc.edu/RePEc/bocode/c)`
- See Ledolter (2013) for details

Random Forests

- Related to the idea of aggregating trees (a.k.a. “ensemble” methods); see Strobl et al. (2009)
- Draw many bootstrap samples of the same size as the full data set, fit a tree to each one, generate predictions, and use the modal prediction across all trees for a given case (use `predict()` for test data)
- We can use the `randomForest()` function in the `randomForest` package of R to grow the trees
- **NOTE: Random Forest methods are extremely computationally intensive for larger data sets**

Random Forests: Methodology

Step 1: Establish the Forest (bagging → bootstrap aggregating)

- Generate $ntree$ bootstrap samples from the original data set, with replacement, and having the same size as the input data set

Step 2: Grow Trees

- For each bootstrap sample from Step 1, grow a classification or regression tree to full size (with **no pruning**) under some set of default parameters
- At each node, **randomly select** predictor variables to use for splitting (difference between random forests and bagging!)

Step 3: Prediction

- After all trees in the forest have been grown, form a prediction for **new data** from each tree in the forest
- Classifications are based on the modes, and regression uses means

Random Forests: Decisions

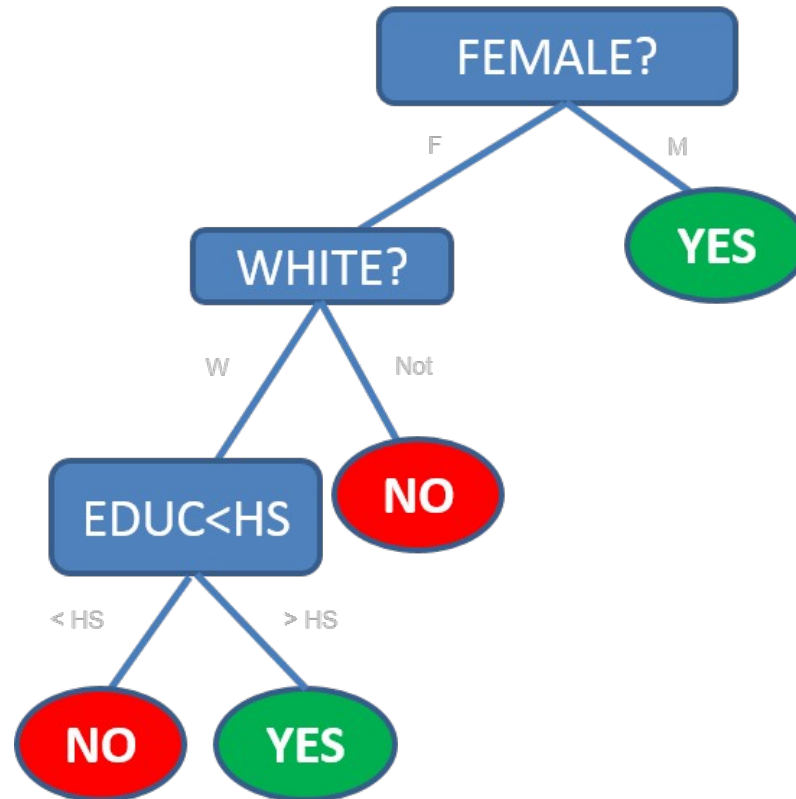
- Key parameters when using random forests:
 - Number of trees (*ntree* argument in `randomForest`)
 - Sizes of the nodes (*nodesize*)
 - Number of variables that will be **randomly considered** at each node (*mtry*)
 - Larger values of *mtry* will increase the strength of the tree, and the correlation between the results of one tree and another in the forest

Random Forests: Performance

- Random Forests automatically generate an internally valid, and in many cases unbiased, assessment of error (OOB error rate)
 - **Step 1:** Each tree in the forest uses approximately 2/3 of the original data for training and the remaining 1/3 of cases for testing. Call these “Out of Bag Predictions.”
 - **Step 2:** For a given case, aggregate all “Out of Bag Predictions” across the roughly $n_{tree}/3$ trees for which the case was OOB.
 - **Step 3:** Compare the aggregated estimate from Step 2 to the actual value of the outcome for each case in the data set
 - Correct/Incorrect for Classification
 - Residual for Regression ($y_i - \text{Step 2 Estimate}$).

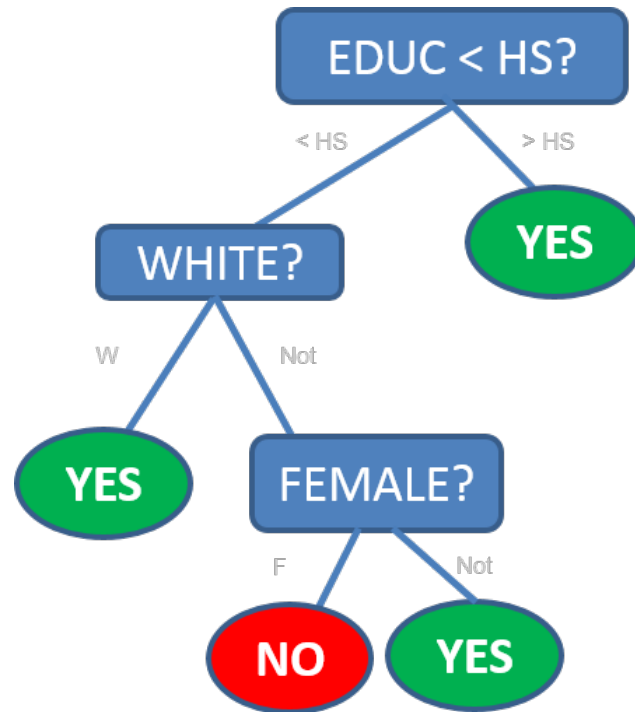
Random Forests: Illustration

First Tree



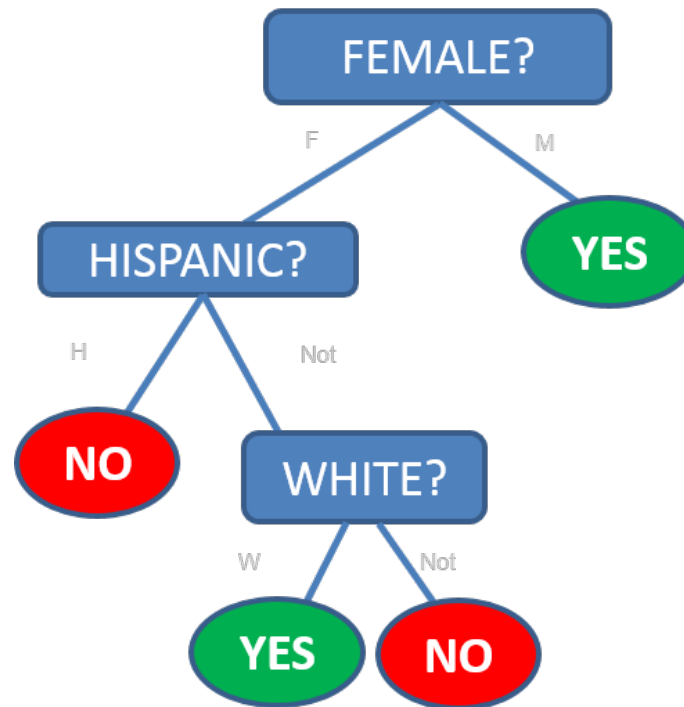
Random Forests: Illustration 1

Second Tree

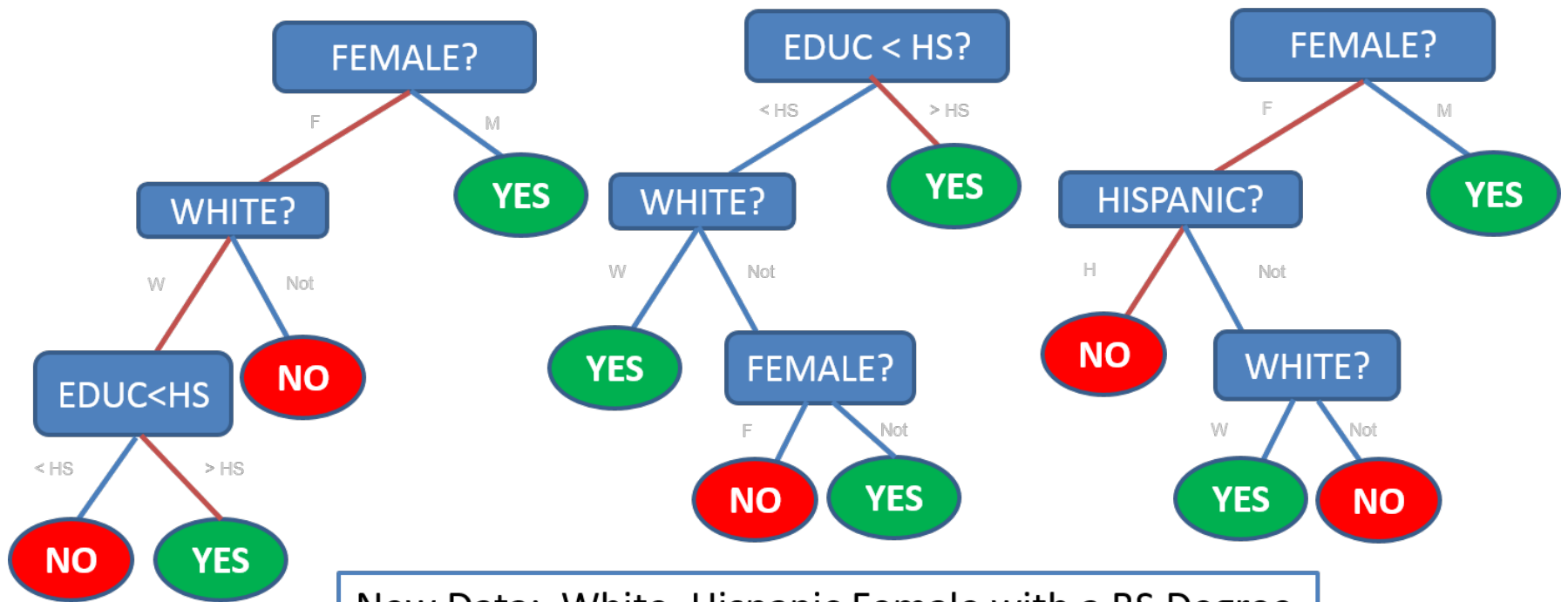


Random Forests: Illustration 2

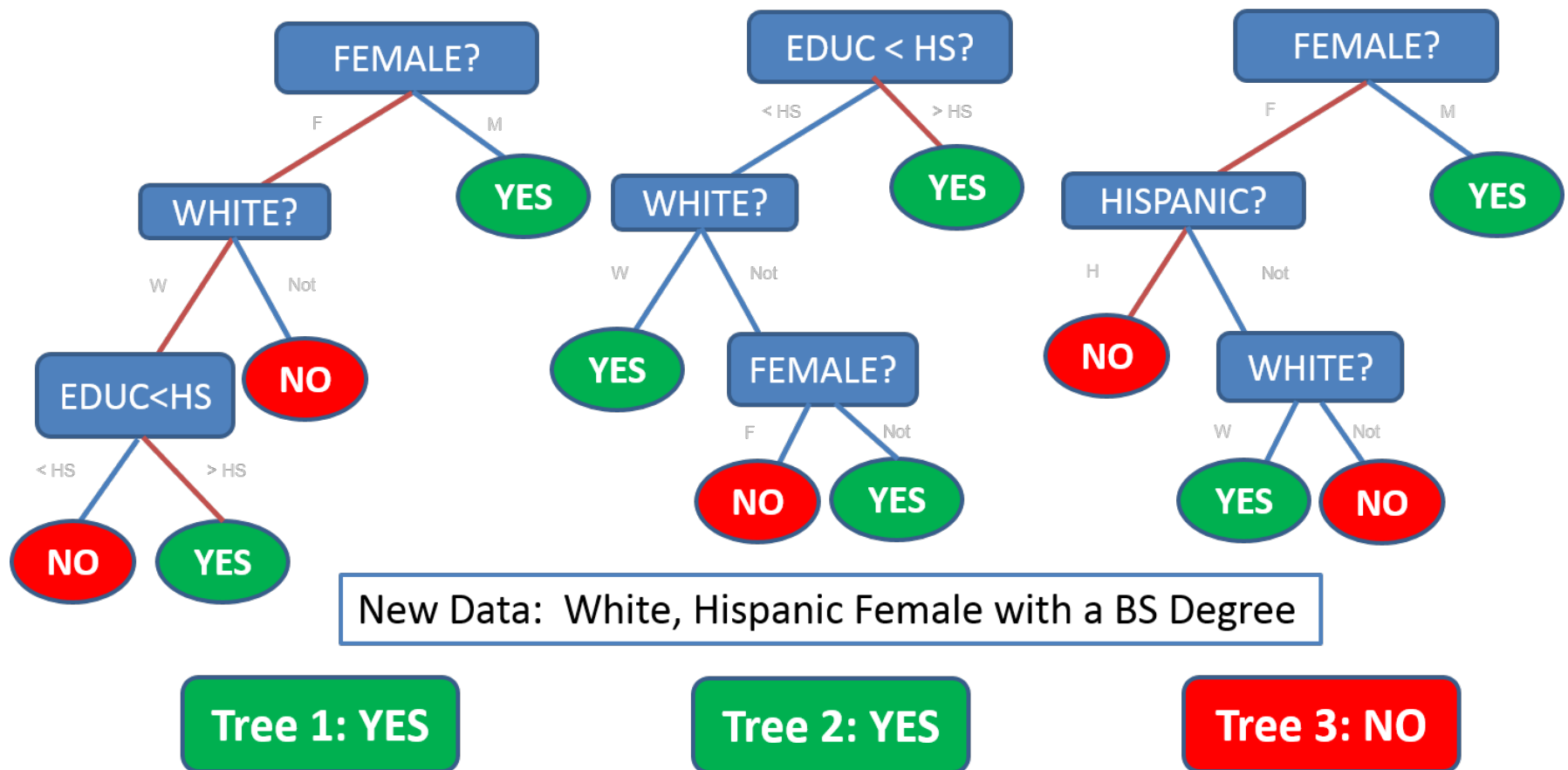
Third Tree



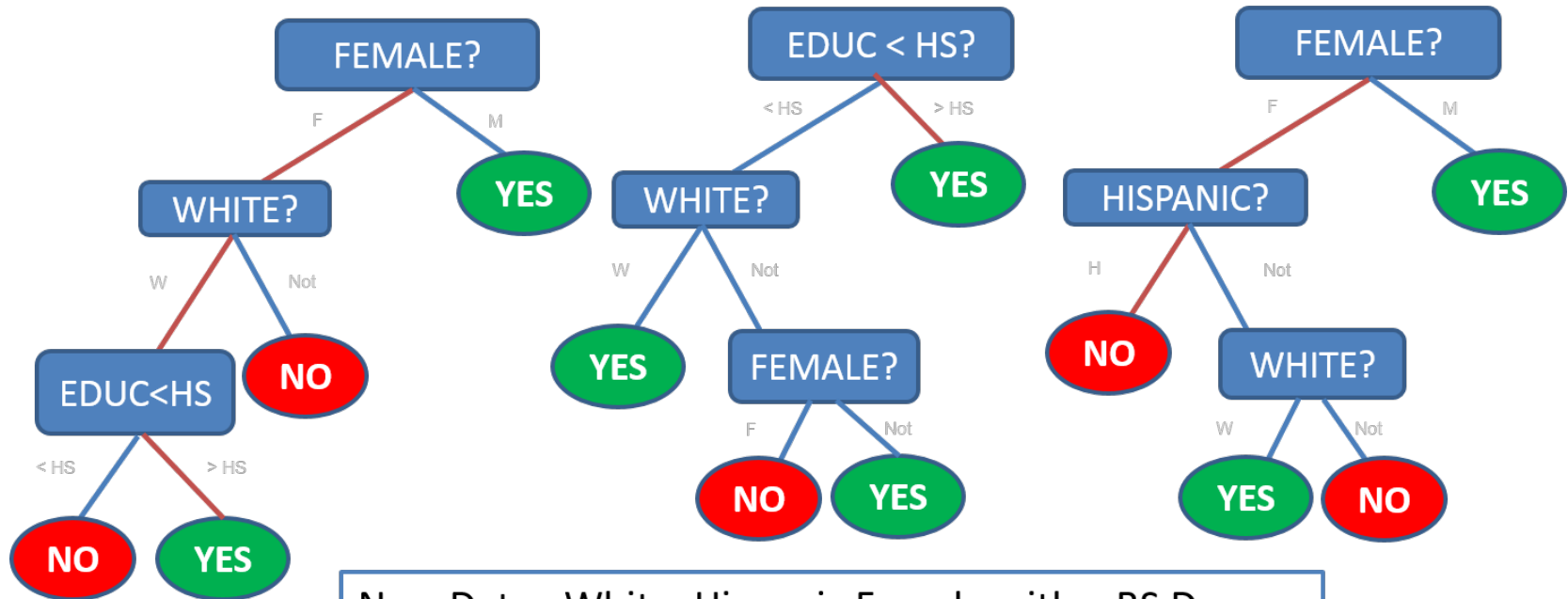
Random Forests: Illustration 3



Random Forests: Illustration 4



Random Forests: Illustration 5



New Data: White, Hispanic Female with a BS Degree

Tree 1: YES

Tree 2: YES

Tree 3: NO

Forest Uses Majority Rule: Prediction for New Data: YES

Predicted Probability of Yes for New Data: .66

Random Forests: Variable Importance 1

- Random Forests do not produce coefficients or significance tests for predictors.
- They do produce **variable importance measures** that can help rank order the utility of the variables in predicting the outcome
 - See Fischer et al. (2017) on Canvas for an example in the context of a simulation study with many parameters varying.
- Variable Importance measures can be biased if the predictor variables are either highly correlated or on different scales:
 - Use **Conditional Forests** instead (e.g. `partykit` package)
 - See: [Additional software for random forests.](#)

Random Forests: Variable Importance 2

- Steps on determining variable importance:
 - Step 1: Obtain OOB error for prediction accuracy
 - Step 2: Randomly permute the values of each variable several times, one variable at a time, and re-calculate OOB
- The VIMP for a variable X is defined as the difference between the perturbed and the unperturbed error rate – averaged over all trees
- The idea is that if the variable is not important (the null hypothesis), then rearranging the values of that variable will not degrade prediction accuracy.

Random Forests: Software

Python: scikit-learn

- R:
- caret
 - **randomforest{randomForest package}**
 - rfsrc{randomForestSRC + ggRandomForest packages}
 - **cforest{partykit package; see help(cforest)}**
 - CoreModel{CORElearn package}

Stata only has user-contributed packages

- SAS has capability through Enterprise Miner and possibly others
- SPSS uses R
- Additional software:

Looking Ahead...

- Next week, we will consider a full application of classification trees for predicting response propensity in Wave 2 of the NESARC
- Classification trees are more often seen as useful tools for this task (especially for nonresponse adjustment purposes...)
- We will also discuss applications of tree-based methods to complex sample survey data